

Лабораторная работа № 6

Контроль версий конфигураций и сценариев с помощью Git

Продолжительность

90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - Git;
 - сценарий `/usr/local/sbin/server-health.sh`, созданный в лабораторной работе № 5;
 - установленный Nginx.
-

1. Цель работы

Научиться использовать Git для контроля изменений в административных сценариях, конфигурациях и документации Linux-сервера.

В ходе работы необходимо:

- создать инфраструктурный Git-репозиторий;
 - различать Working Tree, Staging Area и Repository;
 - подготовить структуру каталогов;
 - настроить `.gitignore`;
 - проверить файлы перед фиксацией;
 - создать несколько commit;
 - сравнить версии файлов;
 - отменить изменения в Working Tree;
 - убрать файл из Staging Area;
 - создать и объединить отдельную ветку;
 - восстановить предыдущую версию файла;
 - проверить отсутствие секретов.
-

2. Основной принцип работы

Безопасный цикл изменения:

Редактирование



`git status`



`git diff`



Техническая проверка



`git add <конкретные файлы>`



`git diff --staged`



`git commit`



Повторная проверка

Git фиксирует изменение файла, но не доказывает, что сценарий или служба работают правильно.

Перед commit выполняется соответствующая проверка:

Тип файла	Проверка
Bash	<code>bash -n</code> , ShellCheck
Nginx	<code>nginx -t</code>
SSH	<code>sshd -t</code>
Markdown	Просмотр содержимого
systemd unit	<code>systemd-analyze verify</code>

3. Структура репозитория

В лабораторной работе создаётся следующая структура:

```
linux-infrastructure
├── configs
│   ├── nginx
│   ├── openvpn
│   └── systemd
├── scripts
│   └── server-health.sh
├── documentation
│   ├── topology.md
│   └── recovery.md
├── examples
│   └── .env.example
├── tests
├── .gitignore
└── README.md
```

Назначение каталогов

Каталог	Назначение
configs	Очищенные конфигурации и шаблоны
scripts	Административные Bash-сценарии
documentation	Топология, адресация и процедуры
examples	Примеры параметров без реальных секретов
tests	Проверки конфигураций и сценариев

Репозиторий создаётся отдельно от /etc.

Нельзя выполнять лабораторные эксперименты с ветками непосредственно внутри действующего каталога конфигурации службы.

4. Базовые задания

Задание 1. Установить и настроить Git

Установить Git:

```
sudo apt update
sudo apt install -y git shellcheck
```

Проверить версии:

```
git --version
shellcheck --version
```

Настроить имя автора:

```
git config --global user.name "Student Name"
```

Настроить учебный адрес электронной почты:

```
git config --global user.email "student@example.test"
```

Проверить:

```
git config --global --list
```

Пример результата:

```
user.name=Student Name
user.email=student@example.test
```

Эти параметры записываются в метаданные каждого нового commit.

Задание 2. Создать репозиторий

Перейти в домашний каталог:

```
cd ~
```

Создать каталог:

```
mkdir -p linux-infrastructure  
cd linux-infrastructure
```

Инициализировать репозиторий с основной веткой main:

```
git init -b main
```

Если команда не поддерживается установленной версией Git:

```
git init  
git branch -M main
```

Проверить текущую ветку:

```
git branch --show-current
```

Ожидаемый результат:

```
main
```

Проверить состояние:

```
git status
```

Пока репозиторий не содержит commit, Git сообщает:

```
No commits yet
```

Задание 3. Подготовить структуру репозитория

Создать каталоги:

```
mkdir -p \  
  configs/nginx \  
  configs/openvpn \  
  configs/systemd \  
  scripts \  
  documentation \  
  examples \  
  tests
```

Скопировать сценарий из предыдущей лабораторной работы:

```
sudo cat /usr/local/sbin/server-health.sh \  
  > scripts/server-health.sh
```

Назначить право выполнения:

```
chmod 750 scripts/server-health.sh
```

Создать очищенную копию конфигурации Nginx:

```
sudo cat /etc/nginx/sites-available/default \  
> configs/nginx/default.conf
```

Создать описание репозитория:

```
nano README.md
```

Добавить:

Linux Infrastructure

Учебный репозиторий конфигураций, сценариев и документации Linux-инфраструктуры.

Состав

- ``scripts/`` – административные Bash-сценарии;
- ``configs/`` – очищенные конфигурации и шаблоны;
- ``documentation/`` – эксплуатационная документация;
- ``examples/`` – примеры параметров без секретов;
- ``tests/`` – технические проверки.

Правила

1. Перед commit выполняется проверка синтаксиса.
2. Реальные пароли, токены и private keys не сохраняются.
3. Live-конфигурации не редактируются через переключение веток Git.
4. Каждый commit должен объяснять назначение изменения.

Создать документ восстановления:

```
nano documentation/recovery.md
```

Добавить:

Восстановление конфигурации

Перед применением конфигурации необходимо:

1. Создать резервную копию действующего файла.
2. Сравнить действующую и подготовленную версии.
3. Проверить синтаксис.
4. Применить конфигурацию через reload или restart.
5. Проверить исходный пользовательский сценарий.

Git не заменяет резервное копирование данных сервера.

Создать пример параметров:

```
nano examples/.env.example
```

Добавить:

```
VPN_USERNAME=CHANGE_ME  
VPN_PASSWORD=CHANGE_ME  
SERVER_ADDRESS=CHANGE_ME
```

Проверить структуру:

```
find . -maxdepth 3 -type f -o -type d | sort
```

Задание 4. Настроить .gitignore

Создать файл:

```
nano .gitignore
```

Добавить:

```
# Редакторы
```

```
*~
```

```
*.swp
```

```
*.swo
```

```
# Резервные копии и дампы
```

```
*.bak
```

```
*.backup
```

```
*.dump
```

```
*.sql
```

```
# Захваты сетевого трафика
```

```
*.pcap
```

```
*.pcapng
```

```
# Секреты и закрытые ключи
```

```
.env
```

```
secrets/
```

```
*.key
```

```
*.pem
```

```
# Временные данные
```

```
tmp/
```

```
cache/
```

```
*.log
```

Проверить работу правил.

Создать тестовые игнорируемые файлы:

```
mkdir -p secrets
touch test.pcap
touch secrets/client.key
touch README.md.bak
```

Выполнить:

```
git status --short
```

Файлы `test.pcap`, `client.key` и `README.md.bak` не должны отображаться как `Untracked`.

Проверить конкретное правило:

```
git check-ignore -v test.pcap
git check-ignore -v secrets/client.key
```

Удалить тестовые файлы:

```
rm -f test.pcap README.md.bak
rm -rf secrets
```

Важно

`.gitignore`:

- помогает избежать случайного добавления файлов;
- не является механизмом защиты;
- не действует на уже отслеживаемые файлы;
- может быть обойдён командой `git add -f`.

Задание 5. Создать первый commit

Начать с проверки состояния:

```
git status
```

Проверить Bash-сценарий:

```
bash -n scripts/server-health.sh
```

Проверить через ShellCheck:

```
shellcheck scripts/server-health.sh
```

Проверить действующую конфигурацию Nginx:

```
sudo nginx -t
```

Проверить исполняемый признак сценария:

```
ls -l scripts/server-health.sh
```

Перед добавлением выполнить поиск потенциальных секретов:

```
grep -RInE \
  --exclude-dir=.git \
  'BEGIN .*PRIVATE KEY|password[[:space:]]*=|token[[:space:]]*=|api[_-
]?key[[:space:]]*=' \
  configs scripts documentation || true
```

Добавить только выбранные файлы:

```
git add \
  .gitignore \
  README.md \
  scripts/server-health.sh \
  configs/nginx/default.conf \
  documentation/recovery.md \
  examples/.env.example
```

Проверить Staging Area:

```
git status
```

Посмотреть содержимое будущего commit:

```
git diff --staged
```

Создать commit:

```
git commit \
  -m "Initialize Linux infrastructure repository"
```

Проверить:

```
git status
git log --oneline --decorate -5
```

Ожидаемое состояние:

```
nothing to commit, working tree clean
```

Задание 6. Исследовать Working Tree и git diff

Открыть README:

```
nano README.md
```

Добавить в конец:

```
## Проверка изменений
```

Перед фиксацией используется последовательность:

```
`status` → diff → test → add → diff --staged → commit`
```

Проверить состояние:


```
git status --short
```

Пример:

```
M README.md
```

Файл уже отслеживается, но изменён в Working Tree.

Посмотреть изменение:

```
git diff -- README.md
```

Подготовить файл:

```
git add README.md
```

Проверить:

```
git status --short
```

Теперь изменение находится в Staging Area:

```
M README.md
```

Сравнить подготовленное состояние с последним commit:

```
git diff --staged -- README.md
```

Создать commit:

```
git commit \  
-m "Document configuration change workflow"
```

Проверить историю:

```
git log --oneline --decorate -5
```

Задание 7. Отменить изменение в Working Tree

Создать ошибочное незакоммиченное изменение:

```
printf '\nУдалять рабочие конфигурации без backup.\n' \  
>> documentation/recovery.md
```

Проверить:

```
git status --short  
git diff -- documentation/recovery.md
```

Перед восстановлением сохранить временную копию:

```
cp -a \  
documentation/recovery.md \  
/tmp/recovery.md.before-restore
```

Отменить незакоммиченное изменение:

```
git restore documentation/recovery.md
```

Проверить:

```
git status --short  
git diff -- documentation/recovery.md  
tail documentation/recovery.md
```

Ошибочная строка должна исчезнуть.

Важно

`git restore` может уничтожить полезные незакоммиченные изменения.

Правильный порядок:

`git status` → `git diff` → резервная копия → `git restore`

Git не сможет восстановить данные, которые никогда не находились в Staging Area или Repository.

Задание 8. Убрать файл из Staging Area

Добавить в `documentation/recovery.md` раздел:

```
cat >> documentation/recovery.md <<'EOF'
```

```
## Проверка после восстановления
```

После восстановления необходимо проверить:

- синтаксис конфигурации;
- состояние службы;
- слушающий порт;
- исходный пользовательский сценарий.

```
EOF
```

Подготовить файл:

```
git add documentation/recovery.md
```

Проверить:

```
git status --short  
git diff --staged -- documentation/recovery.md
```

Убрать файл из Staging Area, сохранив правки:

```
git restore --staged documentation/recovery.md
```

Проверить:

```
git status --short
```

Ожидаемое состояние:

```
M documentation/recovery.md
```

Изменения остались в Working Tree, но не войдут в следующий commit.

После проверки снова подготовить файл:

```
git add documentation/recovery.md
```

Создать commit:

```
git commit \
-m "Add post-recovery verification procedure"
```

Задание 9. Создать ветку и объединить изменение

Создать отдельную ветку:

```
git switch -c improve-health-check
```

Проверить:

```
git branch --show-current
```

Ожидаемый результат:

```
improve-health-check
```

Открыть сценарий:

```
nano scripts/server-health.sh
```

После констант кодов завершения добавить:

```
readonly SCRIPT_VERSION='1.1'
```

В начало функции usage() добавить строку:

```
printf 'server-health.sh version %s\n\n' "$SCRIPT_VERSION"
```

Проверить изменения:

```
git status
git diff -- scripts/server-health.sh
```

Проверить синтаксис:

```
bash -n scripts/server-health.sh
```

Проверить ShellCheck:

```
shellcheck scripts/server-health.sh
```

Проверить справку:

```
./scripts/server-health.sh -h
```

Подготовить файл:

```
git add scripts/server-health.sh
```

Посмотреть будущий commit:

```
git diff --staged
```

Создать commit:

```
git commit \  
-m "Display health check script version"
```

Перейти в основную ветку:

```
git switch main
```

Убедиться, что изменение отсутствует:

```
grep SCRIPT_VERSION scripts/server-health.sh || true
```

Объединить ветку:

```
git merge improve-health-check
```

Проверить результат:

```
grep SCRIPT_VERSION scripts/server-health.sh  
bash -n scripts/server-health.sh  
shellcheck scripts/server-health.sh  
git log --oneline --graph --decorate -10
```

Удалить уже объединённую ветку:

```
git branch -d improve-health-check
```

Вывод

Ветка изолирует изменение в репозитории, но не заменяет отдельный тестовый сервер или snapshot виртуальной машины.

Задание 10. Восстановить предыдущую версию файла

Посмотреть последние commit:

```
git log --oneline --decorate -5
```

Посмотреть изменение последнего commit:

```
git show HEAD
```

Посмотреть предыдущую версию сценария без изменения Working Tree:

```
git show HEAD~1:scripts/server-health.sh |  
head -n 20
```

Сохранить текущий файл:

```
cp -a \  
scripts/server-health.sh \  
/tmp/server-health.sh.current
```

Восстановить предыдущую версию в Working Tree:

```
git restore \  
--source=HEAD~1 \  
scripts/server-health.sh
```

Проверить:

```
git status --short  
git diff -- scripts/server-health.sh  
grep SCRIPT_VERSION scripts/server-health.sh || true
```

Проверить восстановленную версию:

```
bash -n scripts/server-health.sh  
shellcheck scripts/server-health.sh
```

Создать rollback commit:

```
git add scripts/server-health.sh
```

Проверить:

```
git diff --staged
```

Зафиксировать:

```
git commit \  
-m "Restore previous health check script version"
```

Проверить историю:

```
git log --oneline --graph --decorate -10
```

Обратить внимание

Восстановление версии из старого commit создаёт новое изменение в Working Tree.

История старых commit при этом не удаляется.

5. Итоговая проверка репозитория

Проверить состояние:

```
git status
```

Ожидается:

```
nothing to commit, working tree clean
```

Посмотреть историю:

```
git log --oneline --graph --decorate -10
```

Посмотреть последний commit:

```
git show --stat HEAD
```

Вывести отслеживаемые файлы:

```
git ls-files
```

Проверить режим сценария:

```
git ls-files -s scripts/server-health.sh
```

Для исполняемого файла ожидается режим:

```
100755
```

Повторить проверку секретов:

```
grep -RI nE \
  --exclude-dir=.git \
  'BEGIN .*PRIVATE KEY|password[[:space:]]*=|token[[:space:]]*=|api[_-
]?key[[:space:]]*=' \
  configs scripts documentation || true
```

Проверить игнорируемые шаблоны:

```
git check-ignore -v test.pcap 2>/dev/null || true
git check-ignore -v private.key 2>/dev/null || true
```

6. Дополнительные задания

Дополнительное задание 1. Создать и разрешить merge conflict

Создать ветку:

```
git switch -c change-readme-a
```

Изменить первую строку README.md:

Linux Infrastructure Repository

Создать commit:

```
git add README.md
git commit -m "Rename infrastructure repository heading"
```

Вернуться в main:

```
git switch main
```

Изменить ту же строку иначе:

Linux Server Infrastructure

Создать commit:

```
git add README.md
git commit -m "Clarify repository purpose"
```

Попытаться объединить ветку:

```
git merge change-readme-a
```

Git должен сообщить о конфликте.

Открыть файл:

```
nano README.md
```

В нём появятся маркеры:

```
<<<<<< HEAD
# Linux Server Infrastructure
=====
# Linux Infrastructure Repository
>>>>>> change-readme-a
```

Выбрать итоговый вариант, например:

Linux Server Infrastructure Repository

Удалить маркеры конфликта.

Проверить:

```
git diff
```

Подготовить исправленный файл:

```
git add README.md
```

Завершить merge:

```
git commit \
-m "Resolve README heading conflict"
```

Проверить:

```
git status  
git log --oneline --graph --decorate -10
```

Удалить ветку:

```
git branch -d change-readme-a
```

Конфликт считается разрешённым только после проверки итогового файла, а не после удаления маркеров.

Дополнительное задание 2. Создать учебный Remote Repository

Создать локальный каталог, имитирующий Git-сервер:

```
mkdir -p ~/git-server
```

Создать bare repository:

```
git init --bare \  
~/git-server/linux-infrastructure.git
```

Вернуться в рабочий репозиторий:

```
cd ~/linux-infrastructure
```

Добавить remote:

```
git remote add origin \  
~/git-server/linux-infrastructure.git
```

Проверить:

```
git remote -v
```

Отправить ветку:

```
git push -u origin main
```

Проверить связь локальной и удалённой ветки:

```
git branch -vv  
git log --oneline --decorate --all --graph -10
```

Создать аннотированный tag:

```
git tag -a lab6-ok \  
-m "Laboratory work 6 accepted"
```

Проверить:

```
git show lab6-ok
```


Отправить tag:

```
git push origin lab6-ok
```

Вывод

Remote Repository повышает сохранность Git-истории и позволяет обмениваться изменениями, но не является полноценной резервной копией сервера.

Дополнительное задание 3. Создать локальную проверку секретов перед commit

Создать hook:

```
nano .git/hooks/pre-commit
```

Добавить:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
pattern='BEGIN .*PRIVATE KEY|password[[:space:]]*=|token[[:space:]]*=|api[_-]?key[[:space:]]*='
```

```
staged_files=$(  
    git diff --cached \  
        --name-only \  
        --diff-filter=ACMR  
)
```

```
if [[ -z $staged_files ]]; then  
    exit 0  
fi
```

```
if git grep \  
    --cached \  
    -n \  
    -I \  
    -E "$pattern" \  
    -- $staged_files; then  
    printf '%s\n' \  
        'Commit blocked: обнаружен возможный секрет.' >&2  
    exit 1  
fi
```

```
exit 0
```

Сделать исполняемым:

```
chmod 750 .git/hooks/pre-commit
```

Создать тестовый файл:

```
printf 'password=DEMO_VALUE\n' \  
> examples/unsafe-example.conf
```

Добавить:

```
git add examples/unsafe-example.conf
```

Попытаться выполнить commit:

```
git commit \  
-m "Add unsafe example"
```

Commit должен быть заблокирован.

Убрать файл из Staging Area:

```
git restore --staged examples/unsafe-example.conf
```

Удалить:

```
rm examples/unsafe-example.conf
```

Важно

Локальный hook:

- помогает обнаружить часть ошибок;
- не является полной защитой;
- не передаётся автоматически при клонировании;
- может быть обойдён;
- не заменяет review и серверную проверку.

7. Основные команды Git

Состояние

```
git status  
git status --short  
git branch --show-current
```

Сравнение

```
git diff  
git diff --staged  
git diff HEAD  
git show HEAD
```

Подготовка и фиксация

```
git add <file>  
git commit -m "Message"
```

История

```
git log --oneline --graph --decorate -10  
git show <commit>
```

Восстановление

```
git restore <file>  
git restore --staged <file>  
git restore --source=<commit> <file>
```

Ветки

```
git switch -c <branch>  
git switch main  
git merge <branch>  
git branch -d <branch>
```

Remote Repository

```
git remote -v  
git fetch  
git push
```

8. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Структуру репозитория.
3. Различие Working Tree, Staging Area и Repository.
4. Содержимое .gitignore.
5. Результаты `bash -n` и `ShellCheck`.
6. Историю из `git log --oneline --graph`.
7. Пример `git diff`.
8. Пример отмены изменения через `git restore`.
9. Пример использования отдельной ветки.
10. Результат проверки потенциальных секретов.
11. Краткий вывод по работе.

Полное содержимое каталога `.git` в отчёт не добавляется.

9. Чек-лист выполнения

- ☐ Git установлен и настроен.
 - ☐ Создан репозиторий `linux-infrastructure`.
 - ☐ Основная ветка называется `main`.
 - ☐ Создана заданная структура каталогов.
 - ☐ Сценарий `server-health.sh` добавлен в репозиторий.
 - ☐ Исполняемый признак сценария сохранён.
 - ☐ Live-конфигурация отделена от Git-репозитория.
 - ☐ `.gitignore` создан до первого `commit`.
 - ☐ PCAP, дампы и `private keys` игнорируются.
 - ☐ Перед `commit` выполнены проверки.
 - ☐ Использовался `git diff --staged`.
 - ☐ Создано несколько осмысленных `commit`.
 - ☐ Выполнено восстановление Working Tree.
 - ☐ Файл удалялся из Staging Area без потери правок.
 - ☐ Создана и объединена отдельная ветка.
 - ☐ Проверена история Git.
 - ☐ Репозиторий не содержит реальных секретов.
 - ☐ Итоговый Working Tree находится в чистом состоянии.
-

10. Контрольные вопросы

1. Чем Working Tree отличается от Staging Area и Repository?
2. Почему `git commit` не сохраняет автоматически все изменения?
3. Чем отличаются `git diff`, `git diff --staged` и `git show HEAD`?
4. Почему механическое выполнение `git add .` опасно?
5. Чем отличаются `git restore` и `git restore --staged`?
6. Почему `.gitignore` не является защитой секретов?
7. Почему Git-репозиторий не следует размещать непосредственно внутри `/etc`?
8. Почему Remote Repository не является полноценной резервной копией сервера?